Sorting Strategies (Detailed Explanation + Code)

Sorting is a process of arranging data in a specific order, typically ascending or descending.
It helps in faster searching, organizing data, and optimizing performance in algorithms.

Bubble Sort

Idea: Compare adjacent elements and swap if out of order. It continues until the array is sorted.
Time Complexity: $O(n^2)$
Space: O(1)
Stable: Yes
Example:

```
void bubbleSort(int arr[]) {
int n = arr.length;
for (int i = 0; i < n-1; i++) {
for (int j = 0; j < n-i-1; j++) {
if (arr[j] > arr[j+1]) {
int temp = arr[j];
arr[j] = arr[j+1];
arr[j+1] = temp;
}
}
}
}
```

Selection Sort

Idea: Select the smallest element in each iteration and put it in the correct place.
Time: $O(n^2)$
Space: O(1)
Example:

```
void selectionSort(int arr[]) {
for (int i = 0; i < arr.length - 1; i++) {
int min = i;
for (int j = i + 1; j < arr.length; j++) {
if (arr[j] < arr[min]) min = j;
}
int temp = arr[min];
arr[min] = arr[i];
arr[i] = temp;
}
}
```

Insertion Sort

Idea: Insert elements into a sorted portion of the array.
Best for small or nearly sorted data.
Time: $O(n^2)$

Example:

```
void insertionSort(int arr[]) {
for (int i = 1; i < arr.length; i++) {
int key = arr[i];
int j = i - 1;
while (j >= 0 && arr[j] > key) {
arr[j + 1] = arr[j];
j--;
}
arr[j + 1] = key;
}
}
```

Merge Sort

Divide and conquer approach.
Splits the array, sorts each half, and merges.
Time: O(n log n)
Space: O(n)
Example:

```
void mergeSort(int[] arr, int l, int r) {
if (l < r) {
int mid = (l + r) / 2;
mergeSort(arr, l, mid);
mergeSort(arr, mid + 1, r);
merge(arr, l, mid, r);
}
}
```

Quick Sort

Pick a pivot, partition array, sort subarrays recursively.
Average Time: O(n log n)
Worst: $O(n^2)$
Example:

```
void quickSort(int[] arr, int low, int high) {
if (low < high) {
int pi = partition(arr, low, high);
quickSort(arr, low, pi - 1);
quickSort(arr, pi + 1, high);
}
}
```

Data Structures & Algorithms (DSA)

Common questions test your understanding of linked lists, stacks, queues, and arrays.
Focus on space-time trade-offs and optimal solutions.

Reverse a Linked List

```
ListNode reverse(ListNode head) {
ListNode prev = null, curr = head;
while (curr != null) {
ListNode next = curr.next;
curr.next = prev;
prev = curr;
curr = next;
}
return prev;
}
```

Detect Cycle in Linked List

Using Floyd’s Cycle Detection Algorithm:

```
boolean hasCycle(ListNode head) {
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
slow = slow.next;
fast = fast.next.next;
if (slow == fast) return true;
}
return false;
}
```

Design Patterns

These are reusable solutions to common design problems in software engineering.
Core patterns: Singleton, Factory, Observer, Builder, Strategy.

Singleton Pattern

Ensures only one instance exists.

```
public class Singleton {
private static Singleton instance;
private Singleton() {}
public static synchronized Singleton getInstance() {
if (instance == null)
instance = new Singleton();
return instance;
}
}
```

Java 8 Features

Key features:
1. Lambda Expressions
2. Stream API
3. Optional Class
4. Functional Interfaces

Example - Streams and Lambdas

```
List nums = Arrays.asList(1,2,3,4,5);
int sum = nums.stream().filter(n -> n % 2 == 0).mapToInt(n -> n).sum();
```

OOPs Concepts

1. Encapsulation – Data hiding
2. Inheritance – Code reusability
3. Polymorphism – Multiple forms
4. Abstraction – Hiding complexity with interfaces/abstract classes